

Projet de synthèse M1 – Partie ACL

1. Sujet

Le problème d'ordonnancement a pour solution une liste d'affectations de tâches pour chaque machine. On se propose à présent de transformer ce problème en un problème de logistique : une tâche devient une ville à livrer, le p_i (entier > 0) de la tâche représente la quantité à livrer dans la ville $n^o i$ et une machine devient un camion. La solution du problème d'ordonnancement est donc interprétée comme le planning d'une entreprise de transport. On suppose que les camions ont une capacité infinie.

□ Le problème consiste donc à présent à établir le parcours de chaque camion lors de sa tournée de livraison.

Pour cela, nous disposons de plusieurs listes de villes, une pour chacune des treize régions métropolitaines.

Ces listes sont données sous forme de fichiers JSON où chaque ville est définie par son nom, sa latitude et sa longitude.

Une telle liste permet donc de produire une carte routière. Celle-ci est représentée sous la forme d'un graphe complet dont les sommets sont les villes et les arêtes les routes entre les villes. Les routes sont rectilignes.

La liste d'affectations d'une machine est interprétée comme une liste de villes à sélectionner sur la carte. Celles-ci peuvent être choisies aléatoirement.

□ Le programme doit établir le chemin le plus court reliant la liste de villes sélectionnées puis à visualiser la feuille de route obtenue.

Pour résumer :

A partir des données :

1. une liste d'affectations des machines (solution produite par un algorithme demandé dans la partie Algo du projet)
2. une liste de fichiers de villes (un fichier JSON par région)

On construit :

1. une carte routière
2. les tournées des camions
3. la visualisation de la carte et des tournées

Contraintes :

Conformément aux hypothèses du problème d'ordonnancement, une ville ne peut pas être livrée par plusieurs camions.

2. Listes de villes au format JSON

Pour chacune des treize régions métropolitaines, on dispose d'un fichier de villes au format JSON.

Exemple du fichier `GrandEst.json` :

[

```
{ "ville": "Strasbourg", "latitude": 48.58, "longitude": 7.75 },
{ "ville": "Reims", "latitude": 49.26, "longitude": 4.03 },
{ "ville": "Metz", "latitude": 49.12, "longitude": 6.17 },
{ "ville": "Mulhouse", "latitude": 47.75, "longitude": 7.34 },
{ "ville": "Nancy", "latitude": 48.69, "longitude": 6.18 },
{ "ville": "Colmar", "latitude": 48.08, "longitude": 7.36 },
{ "ville": "Troyes", "latitude": 48.30, "longitude": 4.08 },
{ "ville": "Charleville-Mézières", "latitude": 49.77, "longitude": 4.72 },
{ "ville": "Châlons-en-Champagne", "latitude": 48.96, "longitude": 4.36 },
{ "ville": "Thionville", "latitude": 49.36, "longitude": 6.17 },
{ "ville": "Épinal", "latitude": 48.18, "longitude": 6.45 },
{ "ville": "Haguenau", "latitude": 48.82, "longitude": 7.78 },
{ "ville": "Sarrebouurg", "latitude": 48.73, "longitude": 7.05 },
{ "ville": "Sarreguemines", "latitude": 49.11, "longitude": 7.07 },
{ "ville": "Saint-Dié-des-Vosges", "latitude": 48.28, "longitude": 6.95 },
{ "ville": "Verdun", "latitude": 49.16, "longitude": 5.38 },
{ "ville": "Vesoul", "latitude": 47.62, "longitude": 6.15 },
{ "ville": "Bar-le-Duc", "latitude": 48.77, "longitude": 5.16 },
{ "ville": "Briey", "latitude": 49.25, "longitude": 5.93 },
{ "ville": "Forbach", "latitude": 49.19, "longitude": 6.90 }
]
```

3. Graphe (ou carte routière)

Une carte routière est obligatoirement représentée sous forme d'instance de la classe **Graphe<S,T>** (cf. exos_graphe_generique.doc) où les paramètres formels S et T sont remplacés par des types de votre choix. Le langage C++ est donc imposé.

□ On rappelle que S et T peuvent aussi bien être des types simples du langage que des classes aussi complexes que vous le souhaitez.

4. Distances sur la carte

Une route est rectiligne. La longueur de celle-ci est donc la distance *géodésique*, c'est-à-dire la distance parcourue en suivant la courbure de la terre. Le calcul de la distance géodésique entre deux points A et B sur terre n'est pas très difficile à effectuer. Il exploite les propriétés du triangle isocèle (ABC) où C est le centre de la terre. Un rapide calcul de trigonométrie permet de calculer la longueur de l'arc A->B (n'hésitez pas à me poser des questions à ce sujet).

5. Construction du parcours de la tournée de livraison des camions

Il s'agit encore une fois du célèbre problème du voyageur de commerce (*TSP* pour *Travelling Salesman Problem*).

Un camion part d'une ville, passe par toutes les villes de la tournée puis revient à son point de départ.

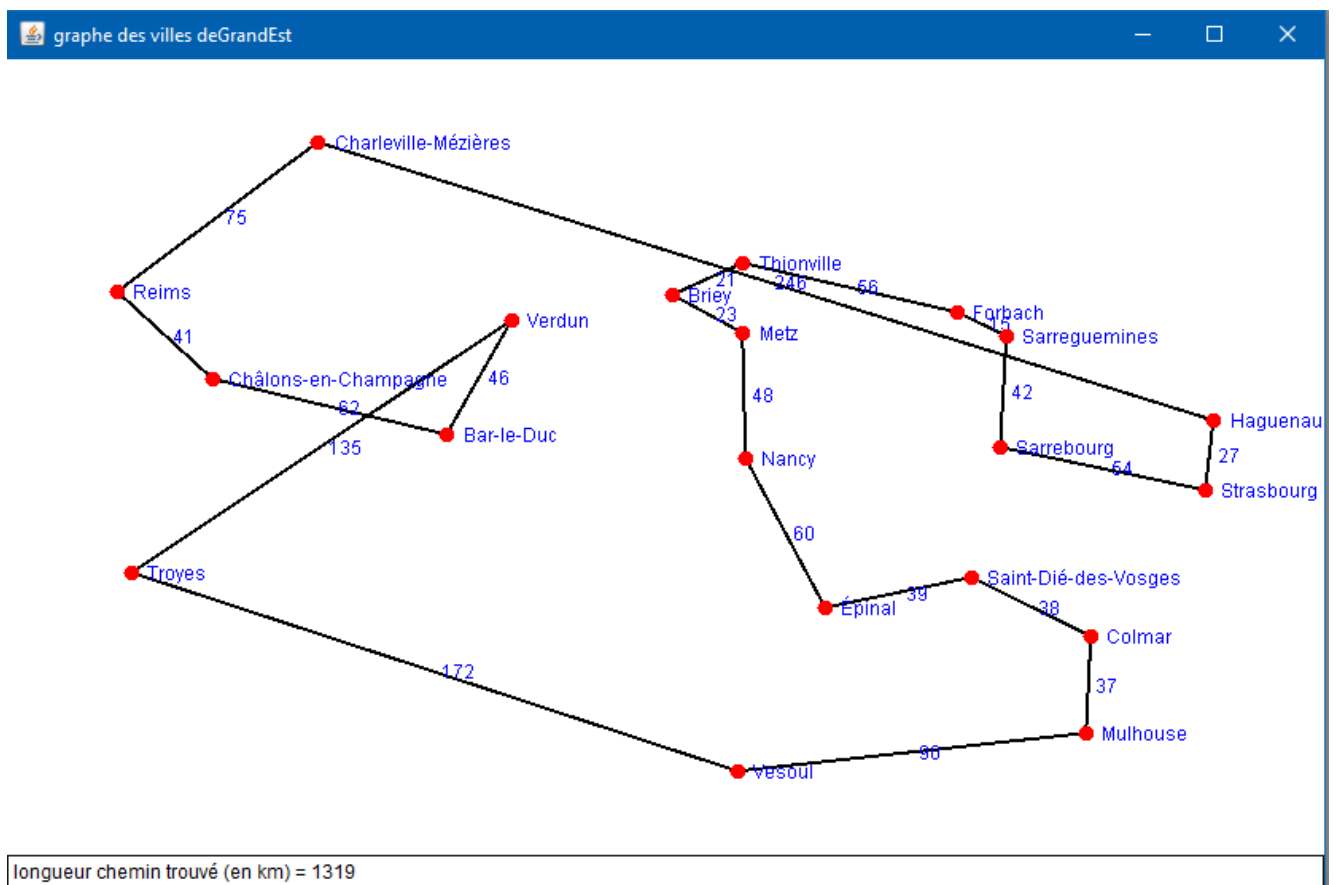
Pour résoudre ce problème, vous pouvez utiliser une des heuristiques que vous avez vues en cours. Pour l'exemple qui suit, une variante de l'algorithme de *Kruskal* a été mise en œuvre. Elle n'est pas très performante mais a le mérite d'être très simple (n'hésitez pas à me poser des questions à ce sujet).

□ Quelque soit l'algorithme que vous choisissiez, sa mise en œuvre doit être **indépendante** du problème considéré, il doit pouvoir fonctionner sur un **Graphe<S,T> quelconque**. Ce point est la grosse difficulté de ce sujet.

6. Visualisation de la carte et du parcours des camions

□ La visualisation de la carte routière et des tournées des camions doivent être réalisées en JAVA.

Exemple de visualisation sur JAVA d'une tournée de l'un des camions incluant les vingt villes du Grand Est extraites du fichier JSON montré au paragraphe 2 :



7. Client JAVA -Serveur C++

Le client JAVA envoie une requête de calcul de chemin (liste de villes) au serveur C++. Celui-ci construit le graphe correspondant à la liste de villes puis calcule la solution au TSP qu'il renvoie au

client. Ce dernier affiche alors la carte et la solution (cf. capture d'écran précédente). La visualisation doit bien sûr résoudre le problème de passage de coordonnées monde-écran. La répartition des tâches est donc strictement imposée : modélisation de graphe et résolution du problème du TSP côté C++ et visualisation côté JAVA. Le client JAVA ne modélise donc pas de graphe ni fait de calcul de chemin. Pour construire un client JAVA-serveur C++, on pourra s'inspirer des exemples fournis avec le sujet (cf. documents du cours de D. MICHEL).

7. Qualité du code source produit

Les notions vues en cours telles que design patterns, templates et foncteurs doivent être exploitées au maximum afin de simplifier le code produit.

Comme toujours, il faut éviter les redondances de code (copiés-collés) (leur présence signifie qu'on a oublié d'écrire une fonction) et préférer aux maladroites instructions *if/else* et *switch*, le mécanisme des fonctions virtuelles ou des templates avec spécialisation.

A l'instar de la classe **Graphe<S,T>**, efforcez-vous d'écrire des classes et des méthodes aussi générales que possible, elles n'en seront que d'autant plus réutilisables.

□ La notation tiendra compte du strict respect de ces consignes.

8. Documents à rendre

L'application complète (sources et exécutables).

9. Date de remise

Tous les documents concernant le projet sont à rendre sur Arche au plus tard le dimanche 4 janvier 2026 à minuit.

10. Soutenance

Les soutenances seront organisées à partir du lundi 5 janvier 2026 à l'UFR MIM.

Le planning des soutenances sera communiqué par courrier électronique la semaine du lundi 15 décembre 2025. Il aura la forme d'un fichier excel partagé qui définira les créneaux disponibles pour placer les soutenances. Les étudiants seront invités à s'y inscrire eux-mêmes pour prendre RDV. Tous les membres de l'équipe devront être présents. La soutenance durera environ 45 minutes.

11. Notation

La notation prendra en compte :

- La qualité du code source et la rigueur de programmation
- Le fonctionnement de l'application